GUIA TÉCNICO OPERACIONAL

Guia de Depuração no Visual Studio 2022

Passo a passo para depurar a API e a Aplicação Web com Hot Reload e dependências no Docker

Este documento apresenta o fluxo de trabalho recomendado para depuração da solução **DesafioBackend**. A arquitetura adotada permite rodar o código C# da **API (.NET 8)** e da aplicação **Web MVC** diretamente no compilador do Visual Studio, mantendo o **PostgreSQL** e o **RabbitMQ** em containers leves no Docker.

> 💡 **Benefícios desta abordagem:** Breakpoints instantâneos com parada imediata de threads, suporte a **Hot Reload** (modificar código sem reiniciar a aplicação), visualização completa de variáveis locais e Janela Imediata para testes em tempo real.

1. Passo a Passo Completo de Depuração

0 Subir Dependências Externas no Docker (PostgreSQL & RabbitMQ)

O seu código C# precisa de um banco de dados e de um broker de mensageria ativo. Abra o terminal (PowerShell ou CMD) na pasta raiz do projeto e execute:

```
docker compose up -d db rabbitmq
```

Esse comando inicializa:

- **PostgreSQL 16:** Porta `5433` (mapeada para a 5432 interna).
- **RabbitMQ 3:** Portas `5672` (protocolo AMQP) e `15672` (painel web de gestão).

1 Abrir a Solução no Visual Studio 2022

1. Abra o **Visual Studio 2022**.
2. Clique em **Abrir um projeto ou solução** (*Open a project or solution*).
3. Navegue até a pasta do projeto e selecione o arquivo:
   `C:\GitHub\Mottu-ops\DesafioBackend\DesafioBackend.sln`

2 Configurar Inicialização Conjunta (API + Web)

Para depurar a API e o Painel Web simultaneamente com um único clique:

1. No painel direito, **Gerenciador de Soluções** (*Solution Explorer*), clique com o **botão direito** sobre o nó raiz: `Solução 'DesafioBackend'`.
2. Selecione **Configurar Projetos de Inicialização...** (*Configure Startup Projects...*).
3. Marque a opção **Vários projetos de inicialização** (*Multiple startup projects*):
   - `DesafioBackend.Api` → Ação: **Iniciar** (*Start*)
   - `DesafioBackend.Web` → Ação: **Iniciar** (*Start*)
   - Os demais projetos de teste e bibliotecas permanecem como *Nenhum (None)*.
4. Clique no botão **Aplicar** e em seguida em **OK**.

3 Colocar Pontos de Interrupção (Breakpoints)

O breakpoint instrui o Visual Studio a pausar o programa exatamente antes de executar uma determinada linha:

1. No Solution Explorer, navegue e abra um arquivo de controle ou serviço:
   Exemplo 1: `src/DesafioBackend.Api/Controllers/MotosController.cs` (método `Cadastrar`)
   Exemplo 2: `src/DesafioBackend.Application/Services/RentalService.cs` (método `CreateAsync`)
2. Clique com o botão esquerdo na **margem cinza à esquerda da linha de código** ou posicione o cursor na linha e aperte a tecla `F9`.
3. Uma **bolinha vermelha** sólida aparecerá, confirmando que o ponto de parada está armado.

4 Iniciar a Sessão de Depuração com F5

1. Pressione a tecla `F5` (ou clique no botão verde com ícone de Play **Iniciar** na barra superior).
2. O Visual Studio compilará a solução e iniciará ambos os projetos:
   - **Swagger da API:** `http://localhost:5000`
   - **Portal Web MVC:** `http://localhost:5001`

5 Inspecionar Valores e Navegar no Código

No navegador, execute a ação correspondente (ex: cadastre uma moto via Swagger ou faça uma locação no Portal Web):

- O Visual Studio piscará na barra de tarefas e pausará a execução na linha destacada em **amarelo**.
- **Inspecionar Valores:** Passe o cursor do mouse sobre variáveis (ex: `request`, `model`, `placa`) para ver os dados em um pop-up flutuante.
- **Janela Locais (Locals):** Exibe todas as variáveis do escopo da função atual com seus respectivos tipos e valores.
- **Janela Imediata (Immediate Window):** Abra via `Depurar > Janelas > Imediata` para executar expressões C# em tempo de execução (ex: digitar `? request.Placa` e pressionar Enter).

2. Tabela de Atalhos de Teclado no Visual Studio

Atalho	Nome do Comando	Finalidade e Comportamento
`F5`	**Continuar (Continue)**	Inicia a depuração ou retoma a execução livre até encontrar o próximo breakpoint.
`F10`	**Passar Sobre (Step Over)**	Executa a linha atual e avança para a linha seguinte **sem entrar** dentro de outros métodos.
`F11`	**Depurar Interrupção (Step Into)**	**Entra dentro** do método chamado na linha atual (ex: entra no Service, Repository ou Handler).
`Shift + F11`	**Sair da Função (Step Out)**	Termina a execução do método atual e volta para o método que fez a chamada original.
`F9`	**Alternar Breakpoint**	Ativa ou remove o ponto de interrupção na linha atual do editor.
`Ctrl + Shift + F9`	**Limpar Breakpoints**	Desativa todos os breakpoints configurados na solução.
`Shift + F5`	**Parar Depuração (Stop)**	Encerra a execução da API e da aplicação Web imediatamente.
`Alt + F10`	**Aplicar Hot Reload**	Aplica as alterações de código feitas no editor diretamente na aplicação em execução.

3. Recursos Avançados de Produtividade

Hot Reload no .NET 8

Durante a depuração no Visual Studio 2022, você não precisa interromper o processo para corrigir uma regra de negócio ou modificar uma tela HTML:

1. Faça a alteração no código C# ou no arquivo `.cshtml`.
2. Clique no ícone de **Foguinho vermelho (Hot Reload)** na barra de ferramentas superior (ou aperte `Alt + F10`).
3. A alteração entra em vigor instantaneamente sem perder a sessão atual do usuário.

Opção Alternativa: Depurar Anexando ao Processo no Docker

Se você preferir executar toda a stack dentro dos containers (via `docker compose up -d`) e anexar o depurador do Visual Studio ao container Linux em tempo de execução:

1. No menu superior do Visual Studio, vá em: **Depurar (Debug)** → **Anexar ao Processo... (Attach to Process...)** ou aperte `Ctrl + Alt + P`.
2. No campo **Tipo de Conexão** (*Connection type*), selecione **Docker (Contêiner do Linux)**.
3. No campo **Destino da Conexão**, clique em **Localizar... (Find...)** e selecione o container desejado:
   - `desafio_backend_api` (para depurar a API REST)
   - `desafio_backend_web` (para depurar a aplicação Web MVC)
4. Na lista de processos que aparecerá abaixo, selecione o processo `dotnet` e clique em **Anexar (Attach)**.
5. Seus breakpoints no código C# serão vinculados e acionados quando requisições chegarem ao container.

4. Tabela de Serviços e Credenciais para Depuração Local

Serviço	Host Local	Porta	Credenciais de Acesso	Finalidade / Detalhes
PostgreSQL 16	`localhost`	`5433`	User: `postgres` Pass: `postgrespassword`	Database: `desafiodb` Connection string em `appsettings.json` da API.
RabbitMQ (AMQP)	`localhost`	`5672`	User: `guest` Pass: `guest`	Broker de mensageria para envio assíncrono de eventos de motos e entregas.
RabbitMQ (Web)	`localhost`	`15672`	User: `guest` Pass: `guest`	Painel interativo para monitoramento de filas e trocas de mensagens.
API Backend	`localhost`	`5000`	User: `admin` Pass: `admin`	Documentação interativa Swagger com suporte a JWT Bearer.
Portal Web MVC	`localhost`	`5001`	User: `admin` Pass: `admin`	Interface gráfica de gestão operacional (motos, entregadores e locações).

Guia de Depuração Concluído
Pronto para execução e testes de qualidade no ambiente de desenvolvimento.